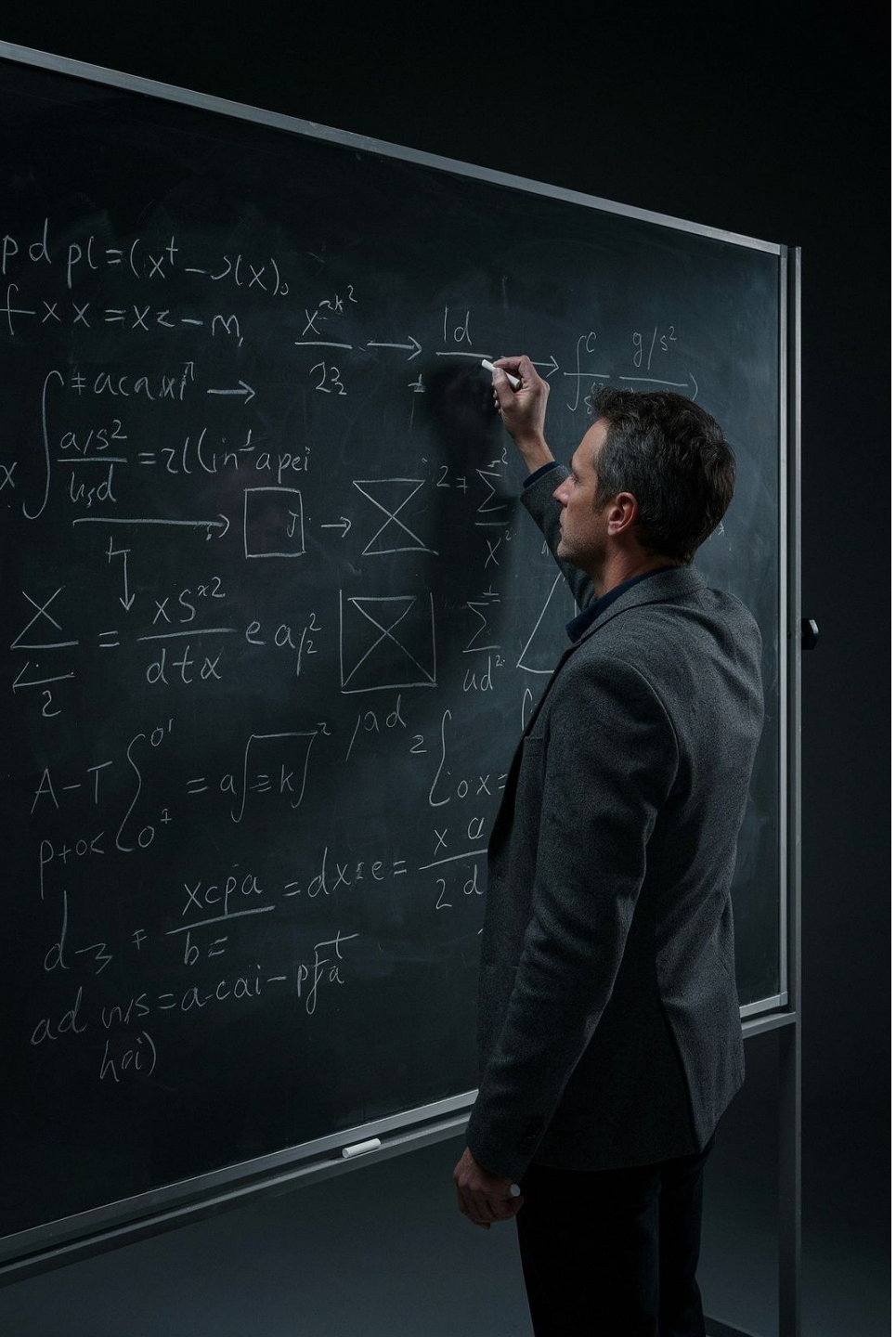
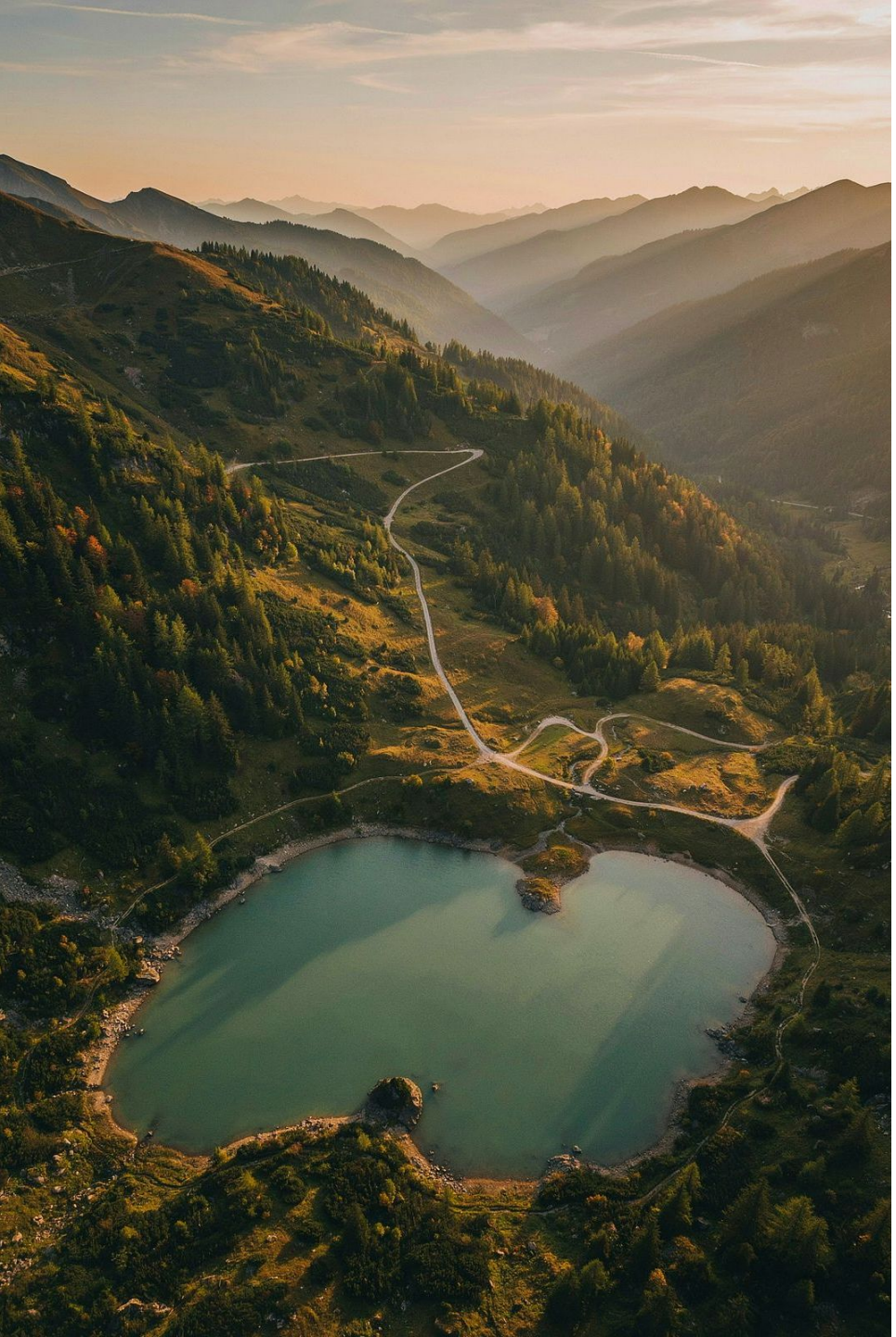


Descenso del Gradiente y Regresión desde Cero

Una exploración profunda de los fundamentos matemáticos del aprendizaje automático: desde la función de coste hasta la implementación completa de regresión lineal, paso a paso y sin atajos.

MÓDULO 5.2 NIVEL INTERMEDIO



¿Por qué optimizar?

Todo modelo de aprendizaje automático tiene un objetivo: **minimizar el error** entre sus predicciones y los valores reales. La optimización es el proceso de encontrar los parámetros que logran ese mínimo. El descenso del gradiente es, con diferencia, el algoritmo más utilizado para conseguirlo.

Error Cuadrático Medio (MSE)

El **Error Cuadrático Medio** es la función de coste estándar en regresión. Mide la media de los cuadrados de las diferencias entre predicciones y valores reales:

$$J(\theta) = \text{MSE} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

¿Por qué cuadrados?

Penaliza errores grandes de forma desproporcionada y produce una función diferenciable en todo su dominio.

¿Por qué la media?

Normaliza el coste respecto al tamaño del conjunto de datos, haciendo comparables los valores entre distintos datasets.

¿Convexo?

Para regresión lineal, el MSE es una función convexa en θ: tiene un único mínimo global, lo que garantiza la convergencia.

Descenso del Gradiente: El Algoritmo

El descenso del gradiente actualiza iterativamente los parámetros moviéndose en la dirección opuesta al gradiente. La regla de actualización es:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad \text{para todo } j$$

1

1. Inicializar

Asignar valores iniciales a θ (normalmente ceros o valores aleatorios pequeños)

2

2. Calcular J y ∇J

Evaluar la función de coste y su gradiente con los parámetros actuales

3

3. Actualizar θ

Aplicar la regla de actualización simultáneamente para todos los parámetros

4

4. Comprobar convergencia

Repetir hasta que ΔJ sea menor que un umbral ε o se alcance el número máximo de iteraciones

Descenso del Gradiente: Paso a Paso

Para regresión lineal con hipótesis $\hat{y} = \theta_0 + \theta_1 x$, las actualizaciones concretas en cada iteración son:

$$\theta_0 := \theta_0 - \alpha \cdot \frac{2}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \cdot \frac{2}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)}$$

Es fundamental actualizar θ_0 y θ_1 **simultáneamente**: calcular ambos gradientes con los valores actuales antes de aplicar ninguna actualización.

En forma vectorial compacta:

Tasa de Aprendizaje (α) y Convergencia

La tasa de aprendizaje **α** es el hiperparámetro más crítico del descenso del gradiente. Controla el tamaño de cada paso en el espacio de parámetros.

α demasiado pequeño

Convergencia muy lenta. El modelo aprende, pero requiere miles de iteraciones para llegar al mínimo.

α óptimo

Convergencia estable y eficiente. El coste decrece suavemente en cada iteración hasta estabilizarse.

α demasiado grande

El algoritmo oscila o diverge. El coste puede aumentar en lugar de disminuir, haciendo el entrenamiento inestable.



Variantes del Descenso del Gradiente

Según cuántos ejemplos se usan para calcular el gradiente en cada paso, existen tres variantes con diferentes compromisos entre precisión y velocidad.



Batch GD

Usa **todo el conjunto de datos** en cada iteración. Gradiente exacto, convergencia suave, pero muy lento con datasets grandes.



Stochastic GD (SGD)

Usa **un solo ejemplo** aleatorio por iteración. Muy rápido, pero con alta varianza: la curva de coste oscila notablemente.



Mini-batch GD

Usa un **subconjunto (batch)** de 32–256 ejemplos. Equilibra velocidad y estabilidad. Es el estándar en deep learning.

Regresión Lineal desde Cero

Implementar regresión lineal en Python puro (con NumPy) consolida la comprensión del algoritmo antes de usar bibliotecas de alto nivel. La estructura básica de la clase:

```
import numpy as np

class LinearRegressionGD:
 def __init__(self, alpha=0.01, n_iter=1000):
 self.alpha = alpha
 self.n_iter = n_iter
 self.theta = None
 self.cost_history = []

 def fit(self, X, y):
 m, n = X.shape
 # Añadir columna de unos (término independiente)
 X_b = np.c_[np.ones((m, 1)), X]
 self.theta = np.zeros(n + 1)

 for _ in range(self.n_iter):
 y_hat = X_b @ self.theta
 error = y_hat - y
 gradient = (2 / m) * X_b.T @ error
 self.theta -= self.alpha * gradient
 self.cost_history.append((error ** 2).mean())

 def predict(self, X):
 X_b = np.c_[np.ones((X.shape[0], 1)), X]
 return X_b @ self.theta
```

Entrenamiento y Diagnóstico Visual

Una vez implementado el modelo, es esencial representar la curva de aprendizaje para verificar que el algoritmo converge correctamente:

```
import matplotlib.pyplot as plt

# Generar datos sintéticos
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X.ravel() + np.random.randn(100)

# Entrenar el modelo
model = LinearRegressionGD(alpha=0.1, n_iter=500)
model.fit(X, y)

# Visualizar convergencia
plt.plot(model.cost_history)
plt.xlabel("Iteración")
plt.ylabel("MSE")
plt.title("Curva de aprendizaje")
plt.show()

print(f"θ₀ = {model.theta[0]:.4f}, θ₁ = {model.theta[1]:.4f}")
```

Con $\alpha=0.1$ y 500 iteraciones, deberías obtener $\theta_0 \approx 4.0$ y $\theta_1 \approx 3.0$, muy próximos a los valores reales del generador de datos.

Comparación con scikit-learn

Nuestra implementación manual debe producir resultados equivalentes a los de `LinearRegression` de scikit-learn (que resuelve el problema analíticamente mediante la ecuación normal). Comparemos:

```
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.preprocessing import StandardScaler

# scikit-learn: solución analítica (ecuación normal)
sk_model = LinearRegression()
sk_model.fit(X, y)
print(f"sklearn → θ₀={sk_model.intercept_:.4f}, θ₁={sk_model.coef_[0]:.4f}")

# scikit-learn: SGD (equivalente a nuestro GD)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
sgd_model = SGDRegressor(max_iter=1000, eta0=0.1, random_state=42)
sgd_model.fit(X_scaled, y)
print(f"SGD     → θ₀={sgd_model.intercept_[0]:.4f}")
```

Ecuación normal (sklearn)

Solución exacta en un paso: [1] Eficiente para datasets pequeños (<10.000 muestras). Coste $O(n^3)$ por la inversión matricial.

Descenso del gradiente

Solución iterativa: escala bien a millones de ejemplos y centenares de características. Imprescindible en deep learning donde la ecuación normal es inviable.

Los 6 Errores Más Frecuentes

1. No normalizar las características

Si las variables tienen escalas muy distintas (ej. edad vs. salario), el gradiente tiene forma elongada y la convergencia es lentísima. **Solución:** aplicar `StandardScaler` antes de entrenar.

2. Tasa de aprendizaje inadecuada

Un α fijo puede ser óptimo al inicio pero subóptimo al acercarse al mínimo. **Solución:** usar un scheduler o buscar α con validación cruzada.

3. Actualización no simultánea de parámetros

Actualizar θ_0 antes de usarlo en el cálculo del gradiente de θ_1 introduce un error sutil pero grave. **Solución:** guardar el gradiente completo antes de actualizar.

4. Olvidar el término independiente (bias)

No añadir la columna de unos a X hace que el modelo deba pasar por el origen. **Solución:** concatenar siempre `np.ones` o usar `fit_intercept=True`.

5. Criterio de parada incorrecto

Detenerse por número fijo de iteraciones puede ser demasiado pronto o demasiado tarde. **Solución:** combinar umbral en ΔJ con número máximo de iteraciones.

6. Confundir MSE con RMSE

El RMSE (raíz del MSE) tiene las mismas unidades que y y es más interpretable, pero su gradiente es más complejo. **Solución:** optimizar MSE y reportar RMSE como métrica.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S05_03_gradiente_regresion — 70 min

Descenso del gradiente y regresión desde cero

sprints/S05/notebooks/S05_03_gradiente_regresion.ipynb